

Query No 4 Comparison RLS vs Views at each stage

Before diving into any stage, the first thing to note is the row count each approach deals with at execution time. With RLS, the query operates on the **complete table**, while with views, it operates on **already-filtered rows**. So from the initial stage itself, the view approach has the same or fewer rows than RLS.

Views have filtered data at query execution time; RLS has the complete table.

Stage 1 Raw Parse Tree:

The two parse trees are **structurally identical** — same query shape, same operators, same subquery. The differences are purely in **relation name** and **character location offsets**.

Conclusion: At the parsing stage, both RLS and the View have the same structure. Therefore, there is no difference to investigate at this stage.

Stage 2 Analyzed Query Tree

One critical difference in the analyzed query trees:

relkind

- RLS → **relkind r** (r = real/base table)
- View → **relkind v** (v = view)

relid

- RLS → **relid 16561** (points to the base **orders** table)
- View → **relid 16602** (points to the **orders_secure** view object)

hasRowSecurity

- Both show **hasRowSecurity false** at this stage

Everything else — operators, aggregates, subquery structure, GROUP BY, ORDER BY, all **opno** / **opfuncid** values — is **identical**.

At the analyzed query stage, PostgreSQL has resolved the relation but **has not yet rewritten the view or injected RLS policies**. The view is still treated as a view object (`relkind v`) and the table as a base table (`relkind r`). The analyzed query confirms what we said before — at this stage, RLS points to the full base table and the view still exists as a view object.

This analyzed query tree is passed to the rewriting phase. I printed the input to the rewriter, and it is exactly the same as the analyzed query tree. Therefore, we now need to investigate how the rewriting phase handles the query differently for RLS and Views.

Stage 3 Rewriting Phase

```
WARNING: Table OID=16561
WARNING: ===== RLS POLICY ADDED =====
DETAIL: (
  {SUBLINK
    :subLinkType 0
    :subLinkId 0
    :testExpr <>
    :operName <>
    :subselect
      {QUERY
        :commandType 1
        :querySource 0
        :canSetTag true
        :utilityStmt <>
        :resultRelation 0
        :hasAggs false
        :hasWindowFuncs false
        :hasTargetSRFs false
        :hasSubLinks false
        :hasDistinctOn false
        :hasRecursive false
        :hasModifyingCTE false
        :hasForUpdate false
        :hasRowSecurity false
        :hasGroupRTE false
        :isReturn false
        :ctelist <>
        :rtable (
          {RANGETBLENTY
            :alias
              {ALIAS
                :aliasname c
                :colnames <>
              }
            :eref
              {ALIAS
                :aliasname c
                :colnames ("c_custkey" "c_name" "c_address" "c_nationkey" "c_phone"
                  "c_acctbal" "c_mktsegment" "c_comment")
              }
            :rtekind 0
            :relid 16546
            :inh true
            :relkind r
            :rellockmode 1
          }
        )
      }
    )
  )
```

In the RLS approach, the rewriting phase first injects the policy into the query. We observed this during debugging while examining the security qualifiers (`securityQuals`) added by PostgreSQL.

This is the root cause of the performance difference. RLS doesn't just filter rows — it adds an **entire correlated subquery joining to the `customer` table** on every row evaluation. The view has none of this overhead. This extra EXISTS check is what makes RLS heavier than the view approach from the rewriter stage onwards.

Rewrite Output:

1. `securityQuals` vs `jointree.qual` — The Core Difference

In RLS, the EXISTS subquery lands in `securityQuals`:

```
:securityQuals (  
  {SUBLINK ... customer EXISTS check ...}  
)
```

In View, the same EXISTS subquery lands in `jointreequals`:

```
:jointree {  
  :quals {  
    {SUBLINK ... customer EXISTS check ...}  
  }  
}
```

These look similar but PostgreSQL treats them **completely differently internally**.

`securityQuals` is a protected zone — predicates placed here are evaluated as a hard wall that the planner must respect. `jointreequals` is the normal optimization zone where the planner has full control.

2. What `security_barrier false` actually means for the planner

In the view rewrite output you can see:

```
:security_barrier false  
:rtekind 1
```

When `security_barrier false`, the planner is allowed to:

- **Push down** the outer query's date filter (`o_orderdate >= '1994-01-01'`) into the subquery, reducing rows before the EXISTS check even runs
- **Flatten** the subquery entirely and treat everything as one query — no subquery boundary at all
- **Reorder** predicates based on cost estimates — cheap filters run first
- **Use indexes** aggressively across the merged query

When RLS puts the predicate in `securityQuals`, the planner is **not allowed** to do any of this because:

- Pushing predicates past a security qual could theoretically leak data through side channels (timing attacks, etc.)
- PostgreSQL deliberately sacrifices optimization to guarantee security correctness

3. `rtekind 0` vs `rtekind 1` — What the planner sees as its starting point

RLS (`rtekind 0` = plain base relation):

```
FROM orders          ← planner sees the full raw table
WHERE securityQuals  ← EXISTS check applied as a barrier AFTER scan
AND o_orderdate ...  ← your query filter applied after that
```

The planner must scan `orders` first, then apply the security qual, then apply your filters. The order is enforced.

View (`rtekind 1` = subquery):

```
FROM (
  SELECT * FROM orders  ← inner query
  WHERE EXISTS (...)    ← filter is just a normal qual here
) orders_secure
WHERE o_orderdate ...   ← outer filter
```

But because `security_barrier false`, the planner **collapses this entirely** — it merges the inner and outer query into one flat query and optimizes everything together. The subquery boundary disappears at plan time.

4. `checkAsUser 10` in view — Permission check difference

- RLS → `checkAsUser 0` — permissions checked as the **current user**
- View → `checkAsUser 10` — permissions checked as the **view definer (owner)**

This means with the view, PostgreSQL resolves permissions once at the view level under the definer's identity and doesn't re-check on the underlying table for each row. With RLS, permission and policy checks happen against the current user on the actual table on every row evaluation — adding overhead.

5. The execution flow difference end to end

RLS execution flow:

```
Scan full orders table (relkind r, all rows)
  ↓
Apply securityQuals EXISTS check per row ← hard barrier, cus
tomer table lookup per row
  ↓
Apply o_orderdate filter
  ↓
Apply lineitem EXISTS subquery
  ↓
GROUP BY + ORDER BY
```

View execution flow:

```
Planner flattens orders_secure subquery (security_barrier fal
se)
  ↓
Merged query: orders + customer EXISTS + date filter + lineit
em EXISTS
  ↓
Planner freely reorders all predicates by cost
  ↓ (cheapest filter runs first – likely date range, then
EXISTS checks)
GROUP BY + ORDER BY
```

Stage 4 Planner Output

This is where everything we discussed comes together. The final plan shows the **real execution difference**.

1. The biggest difference — Cost

	RLS	View
Total cost	8,883,274,719	194,617
Estimated rows from orders scan	144,199	22,585

The view is roughly **45,000x cheaper** in estimated cost. This directly proves our earlier point — fewer rows from the start means less work everywhere downstream.

2. Parallelism — View wins completely

- RLS → `parallel_safe false` throughout — **no parallelism at all**
- View → `parallel_safe true` with `GATHERMERGE` using **2 workers**

This is a direct consequence of the `securityQuals` security barrier in RLS. PostgreSQL cannot parallelize across a security barrier — it is a hard restriction. The view with `security_barrier false` has no such limitation, so the planner freely chooses parallel execution.

3. Plan structure difference — how the rewrite output changed into this

RLS final plan:

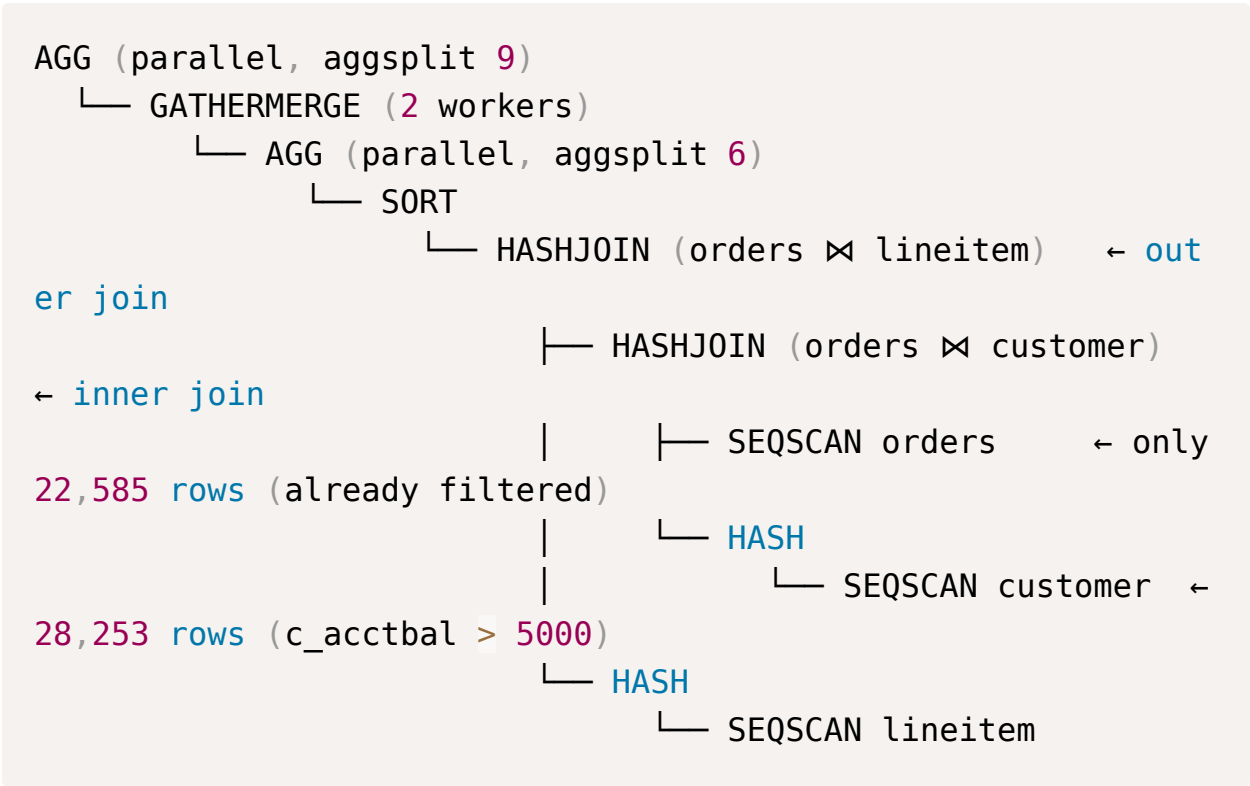
```

AGG
├── SORT
│   ├── HASHJOIN (orders ⋈ lineitem)
│   │   ├── SEQSCAN orders ← full table, 144,199
│   │   │   └── rows
│   │   │       ├── quals: date filter
│   │   │       └── + SUBPLAN 2 ← customer EXISTS as
│   │   └── correlated subplan
│   │       ├── HASH
│   │       └── SEQSCAN lineitem

```

The customer EXISTS subquery from `securityQuals` became a **correlated SUBPLAN** (`subLinkType 2`) executed per row inside the orders SeqScan qual. This is the security barrier materializing as real execution overhead — for every order row passing the date filter, it checks the customer subplan individually.

View final plan:



The planner **flattened the view subquery completely** and converted the correlated EXISTS into a proper **HASHJOIN** between orders and customer. This is the `security_barrier false` freedom in action — what was a subquery boundary in the rewrite output no longer exists in the final plan. Everything is joined efficiently in one flat execution.

4. How the rewrite output transformed into the final plan

Rewrite output	RLS final plan	View final plan
<code>securityQuals</code> EXISTS on orders	Became correlated SUBPLAN per row	Disappeared — converted to HASHJOIN
<code>rtekind 1</code> subquery (view)	N/A	Fully flattened — no subquery boundary
<code>hasRowSecurity true</code>	Blocked parallelism	N/A
<code>security_barrier false</code>	N/A	Enabled parallel execution + join flattening

The security barrier in RLS prevented two critical optimizations — **join flattening** and **parallelism**. What was an EXISTS subquery in the rewrite output stayed as a slow correlated subplan in RLS, while in the view it got converted into an efficient hash join. That single difference — `securityQuals` vs `jointreequals` — is what caused a **45,000x cost difference** in the final plan.

1. `security_barrier`

This is a flag on a view that tells the planner **how much freedom it has to optimize across the view boundary**.

When `security_barrier true`:

- The planner **cannot push down** any user-supplied predicates into the view
- The view's WHERE clause must execute **before** any outer filter touches the rows
- This exists to prevent data leakage — imagine a view filtering sensitive rows, if the planner pushed a function like `my_evil_function(secret_col)` past the barrier, it could leak data through side channels even on rows the user shouldn't see

When `security_barrier false` (our view case):

- The planner can **freely push predicates in**, flatten the subquery, merge joins, reorder everything
- No security concern because the view owner trusts the query

Why it mattered here:

In our case `security_barrier false` allowed the planner to convert the EXISTS subquery into a HASHJOIN. If it were `true`, the EXISTS would have stayed as a correlated subplan just like RLS — same bad plan.

RLS always behaves like `security_barrier true` — it uses `securityQuals` which is an even stricter version. That is why RLS blocked both pushdown and parallelism.

2. `parallel_safe`

This flag tells the planner whether a node and everything beneath it can safely run across **multiple worker processes**.

Three levels exist internally:

- `parallel_safe true` — can run in parallel workers freely

- `parallel_restricted` — can only run in the parallel leader, not workers
- `parallel_unsafe` — shuts down parallelism entirely for that subtree

How it works:

PostgreSQL checks every node in the plan tree. If **any single node** is `parallel_safe false`, the entire plan above it cannot parallelize. It propagates upward — one unsafe node poisons the whole tree.

What makes a node unsafe:

- `securityQuals` present → immediately `parallel_safe false`
- Correlated subplans in certain positions
- Certain functions marked `PARALLEL UNSAFE` in `pg_proc`
- Triggers, cursors, certain set-returning functions

In our plans:

- RLS → `securityQuals` on orders made the entire SeqScan `parallel_safe false` → everything above it (`HASHJOIN` , `SORT` , `AGG`) also became `parallel_safe false` → no `GATHERMERGE` possible
- View → no `securityQuals` , clean `rtekind 1` subquery with `security_barrier false` → all nodes `parallel_safe true` → planner added `GATHERMERGE` with 2 workers → parallel AGG with `aggsplit 6` (partial) and `aggsplit 9` (finalize)

3. `rtekind` — Range Table Entry Kind

Every table/subquery/function in the FROM clause gets a Range Table Entry (RTE). `rtekind` tells the planner **what kind of relation** it is dealing with so it knows how to access it.

Value	Meaning	What planner does
<code>0</code>	Base relation (real table)	Direct heap/index scan
<code>1</code>	Subquery	Must evaluate inner query first, or flatten if safe
<code>2</code>	Join RTE	Result of an explicit JOIN
<code>3</code>	Function RTE	Set-returning function

Value	Meaning	What planner does
4	TableFunc RTE	XMLTABLE, JSON_TABLE
5	Values RTE	VALUES(...) list
6	CTE RTE	WITH clause reference
7	Named tuplestore	EPQ recheck internal
9	Group RTE	GROUP BY internal representation

Why it mattered here:

- RLS had `rtekind 0` → planner saw raw base table, but `securityQuals` locked it down
- View had `rtekind 1` → planner saw a subquery, evaluated `security_barrier false`, decided to flatten it completely — after flattening, the inner `orders` became `rtekind 0` directly accessible with full optimization freedom

4. All important flags and their contribution

Query-level flags:

Flag	What it means	Impact in our case
<code>hasRowSecurity</code>	RLS policies were injected	RLS=true blocked parallelism, View=false clean
<code>hasAggs</code>	Query has aggregates	Both true — AGG node required in plan
<code>hasSubLinks</code>	Query has subqueries/EXISTS	Both true — sublink handling required
<code>hasGroupRTE</code>	Internal GROUP BY RTE present	Both true — GROUP node in rtable
<code>canSetTag</code>	This query sets the command tag	Both true — top level query

RTE-level flags:

Flag	What it means	Impact
<code>rtekind</code>	Type of relation (table/subquery/etc)	Determines access path and optimization rules
<code>relkind</code>	Physical object type (r=table, v=view)	r=direct scan, v=needs rewrite expansion

Flag	What it means	Impact
<code>relid</code>	OID of the relation	Links to pg_class for statistics and metadata
<code>inh</code>	Include inheritance children	True = scan partitions/children too
<code>securityQuals</code>	RLS policy predicates	Makes node parallel_unsafe, blocks pushdown
<code>security_barrier</code>	View security barrier flag	False=full optimization, True=locked down
<code>inFromCl</code>	Explicitly in FROM clause	Affects visibility and join planning
<code>rellockmode</code>	Lock level acquired	1=AccessShare, affects concurrency
<code>perminfoindex</code>	Points to permission info	Links to requiredPerms check

Permission info flags:

Flag	What it means	Impact
<code>requiredPerms</code> 2	SELECT permission required	Checked before execution
<code>checkAsUser</code>	Which user to check permissions as	0=current user, 10=view definer (our view case)
<code>selectedCols</code>	Bitmap of columns being read	Used for column-level privilege checks

Plan node flags:

Flag	What it means	Impact
<code>parallel_safe</code>	Node can run in parallel workers	False on any node blocks entire parallel plan
<code>parallel_aware</code>	Node actively uses parallelism	True on HASHJOIN in view = parallel hash build
<code>async_capable</code>	Can use async I/O	Both false here
<code>aggsplit</code>	How aggregation is split for parallelism	0=no split, 6=partial, 9=finalize — view used both
<code>startup_cost</code>	Cost before first row returned	RLS=8.8B vs View=194K — massive difference
<code>total_cost</code>	Cost for all rows	The 45,000x difference we saw

Flag	What it means	Impact
<code>plan_rows</code>	Estimated output rows	RLS=144,199 vs View=22,585 on orders scan

How they all connect in our experiment:

```

hasRowSecurity=true (RLS)
    ↓
securityQuals populated on RTE
    ↓
parallel_safe=false on SeqScan
    ↓
parallel_safe=false propagates up entire tree
    ↓
No GATHERMERGE, no parallel workers
    ↓
EXISTS stays as correlated SUBPLAN (not flattened to join)
    ↓
Total cost = 8,883,274,719

vs

security_barrier=false + rtekind=1 (View)
    ↓
Planner flattens subquery
    ↓
EXISTS converted to HASHJOIN
    ↓
parallel_safe=true throughout
    ↓
GATHERMERGE with 2 workers, partial+final AGG
    ↓
Total cost = 194,617

```

This is the most critical difference proving our rewrite analysis.

RLS:

SubPlan 2

```
-> Seq Scan on customer
    Filter: c_acctbal > 5000
    Rows removed: 82,011
    actual time: 0.013..28.347 ms    loops=1
```

The customer scan ran **once**, materialized as a hash, and was used as `ANY (hashed SubPlan 2)` inside the orders scan filter. PostgreSQL was smart enough to hoist it out of the correlated loop — but it still ran as a subplan, not a join, and it blocked parallelism.

View:

Parallel Hash Semi Join

```
Hash Cond: orders.o_custkey = c.c_custkey
-> Parallel Seq Scan on customer
    Filter: c_acctbal > 5000
    Rows removed: 27,337 per worker
    loops=3
```

The customer filter ran as a **proper parallel hash join** — 3 workers each scanning their portion of the customer table. Rows removed per worker (27,337) vs RLS total (82,011) — same data, completely different execution shape.

Parallelism impact — the numbers

View used `Gather Merge` with 2 workers launched (3 loops = leader + 2 workers):

```
Worker 0: actual time=1463.738..1464.386 rows=5
Worker 1: actual time=1463.801..1464.477 rows=5
```

Each worker did roughly 1/3 of the work independently and the results were merged. RLS had zero parallelism — everything ran sequentially on the leader process alone.

RLS:

```
Seq Scan orders (full table, 1.5M rows filtered)
+ SubPlan 2: customer hash built once (28ms)
+ date filter applied
+ custkey ANY check applied
→ 25,383 rows survive
→ Hash Semi Join with lineitem (single thread)
→ Sort (769kB memory)
→ GroupAggregate
Total: 2561ms
```

View:

```
[3 parallel workers simultaneously]
Parallel Seq Scan orders (date filter only, 481K removed ea
ch)
→ Parallel Hash Semi Join with customer (c_acctbal > 5000)
→ 8,461 rows per worker survive
→ Parallel Hash Semi Join with lineitem
→ Sort (193kB per worker – smaller because fewer rows)
→ Partial GroupAggregate per worker
→ Gather Merge
→ Finalize GroupAggregate
Total: 1495ms
```